

expected_red

scripts/lib/expected_red.sh — the EXPECTED-RED declaration mechanism for bash suites

Revision: 3 **Last modified:** 2026-08-27T01:26:19Z **Status:** active **Item:** BOB-221 (invariant 30 has no concept of a suite that is supposed to fail)

Overview

Pre-build invariant 30 (CM-BASH-UNIT-TESTS-EXECUTED) asserts “*no bash suite fails*”. The invariant it should assert is “*no bash suite fails unexpectedly*”. Those two differ the moment the constitution’s own test-first discipline is followed:

- §11.4.224 requires the RED test to be written and **observed failing** before the fix exists.
- §11.4.135 requires that guard to **persist** afterwards.

So a correctly-authored, still-open RED is a permanent, *intended* failure — and before BOB-221 the gate counted it as a defect and refused the build. That is a §11.4.120 wrong-seam refusal and a §11.4.201(1) FAIL-bluff: following the constitution made the constitution’s own blocking gate refuse.

scripts/lib/expected_red.sh supplies the missing concept: a way to **declare** that a named suite is supposed to fail, honoured **only** while the declaration is corroborated, current, and accurate — and which **blocks** the moment it stops being any of those.

Parity, not a new bar

The Python side already runs this discipline via
`pytest.mark.xfail(strict=True, reason=...):`

Suite	Item
tests/scaling/test_scaling_envelope.py	BOB-167
tests/stress/ test_plugin_parsers_stress_chaos.py	rutracker ReDoS

`strict=True` is the load-bearing half: an **XPASS** (a declared-red test that unexpectedly passes) *fails*, which forces the marker’s removal. A declaration that never expires is a permanent bluff licence. The bash side is held to the same self-clearing bar — this closed a parity gap, it did not invent a policy.

This is NOT quarantine

	BASH_TEST_QUARANTINE	EXPECTED-RED declaration
Suite is executed	No — excluded entirely	Yes — always run
Verdict is observed	No	Yes — the exit code is checked
Distinguishes “supposed to fail” from “broken”	No	Yes
Self-clears	No	Yes (P3, P4, P7)

Solving BOB-221 by widening quarantine would have traded a false refusal (§11.4.201(1)) for a false null (§11.4.201(6)) — strictly the worse outcome, because a quarantined suite’s failure is never seen at all.

The row format: a repo-relative PATH, exactly two fields

tests/unit/test_ownership_gid_agreement.sh BOB-207

Not a basename. The gate’s glob spans **three** directories — tests/unit, tests/pre_build, tests/hooks — so a basename does not identify a file.

This was the round-1 independent review’s **BLOCKING** finding, and it was reproducible end-to-end through the real extracted invariant-30 loop with one row test_zz_bt.sh BOB-207:

scenario	before (basename key)	after (path key)
two same-basename failing suites (tests/unit + tests/hooks)	RAN=2 FAILED=0 HONOURED=2	RAN=2 FAILED=1 HONOURED=1
the declared suite deleted , the never-reviewed sibling remains	RAN=1 FAILED=0 HONOURED=1	RAN=1 FAILED=2 HONOURED=0

The attack the basename key allowed: a reviewed row exists for tests/unit/test_foo.sh; an author later adds a **new** failing suite at tests/hooks/test_foo.sh and copies the one-line marker into it. It is honoured with **zero** table review — the author-side act alone silences a failure, which is precisely the producer-alone channel §11.4.240 / §11.4.249 exist to close. The basename no longer reaches the decision at all: expected_red_verdict takes the suite **path** and matches on \$ {path#\${PROJECT_ROOT}}/}.

A row that is not a repo-relative path under tests/ (a bare basename, an absolute path, anything containing ..) is **refused**, and so is a row carrying more than two fields.

The eighteen properties

Fixed by BOB-221 and by the round-1 and round-3 independent reviews; asserted by tests/pre_build/test_bob221_expected_red_declaration.sh:

ID	Property
P1	A declared red that fails does not block.
P2	An undeclared failing suite still blocks . (<i>load-bearing</i>)
P3	

ID	Property
	A declared red whose tracked item is closed blocks, naming the rot.
P4	A declared red that passes blocks (strict-xfail parity).
P5	A declaration not corroborated by the file's own marker blocks.
P6	A declaration binds to the file — a same-basename suite in another globbed directory is not excused.
P7	A row that matched no executed suite this run blocks.
P8	A declared red that aborts (exit 3) blocks.
P9	A declared red whose item carries any of §11.4.33's four terminal statuses blocks as stale.
P10	A row with more than two fields blocks.
P11	An item whose stored status is empty blocks <i>without</i> being asserted CLOSED.
P12	A declared suite carrying no marker at all blocks, naming the two-place agreement.
P13	The path match is exact : a row for tests/unit/x.sh does not excuse tests/unit/x.sh.sh.
P14	An inline / trailing # EXPECTED-RED: mention is not a marker — the left anchor is load-bearing.
P15	A bare-basename row is refused by the sweep when it matches nothing.
P16	An unmatched row with extra fields is refused by the sweep.
P17	

ID	Property
	An unmatched row with a malformed item id is refused by the sweep.
P18	Two rows naming one path block as ambiguous, counted once .

P12–P18 were added in round 3, each closing a §11.4.115(F) instrumentation gap: a clause that was correct in the shipped code but whose reviewer-authored mutation survived the suite is unvalidated instrumentation *for that clause*. **P13 is the load-bearing addition** — a prefix-match regression (`[[ "${rel}" == "${rkey}"* ]]`) silently re-opened the round-1 BLOCKING finding (RAN=2 FAILED=0 HONoured=2 for an attacker-added `tests/unit/test_zz_p.sh.sh`) while all twelve prior properties stayed green. P6 cannot see it: a prefix match still blocks a same-basename file in a *different* directory, so the sibling has to share the declared path’s prefix.

P2 is the one that matters most. A mechanism that quietly let any failure through would be a worse defect than the false refusal it was built to fix.

The table has its own freshness contract

P3 (item closed) and P4 (XPASS) are only ever evaluated **when the declared suite runs**. A row whose suite was deleted, renamed, quarantined, or is self-recursive is therefore **never judged**: it can neither stale out nor XPASS out, so it rots silently while staying **armed**.

The post-loop sweep (`expected_red_unmatched_rows`, wired in `pre_build_verification.sh` between the suite loop and the `# END-INVARIANT-30-SUITE-LOOP` marker) closes that: **every row must have been evaluated this run**, or it blocks, naming the row. Measured before the sweep existed: a row naming an absent suite, alongside two real failures, produced RAN=2 FAILED=2 — the ghost row neither blocked nor warned.

The sweep emits **nothing** for an empty table, so the empty-table path stays byte-identical to the pre-BOB-221 rule.

It also **skips rows whose key matched an executed suite**, because those were already judged by `expected_red_verdict` — including any shape refusal. Without that skip a matched malformed row is counted

twice, and the gate prints 2/1 bash unit/pre_build test(s) FAILED: fail-closed, but arithmetic that cannot be true is a §11.4.6 imprecision in the operator’s own message.

Two-place agreement (§11.4.240 / §11.4.249)

Silencing a failure requires **two independent places to agree**:

1. **A row** in BASH_TEST_EXPECTED_RED (in this library) — reviewed at the gate.
2. **An in-source marker** in the suite itself:

```
# EXPECTED-RED: BOB-207
```

Neither the suite author alone nor the table alone can silence anything.

- **A marker with no table row** is **inert** — the table is the authority.
- **A table row with an absent or mismatched marker blocks**.

The marker is matched with a line-anchored pattern

```
(^[[:space:]]*#[[:space:]]*EXPECTED-RED:
```

```
[[:space:]]*<ID>[[:space:]]*$), so prose that merely mentions the marker — including this document — is not a carrier (§11.4.201(7)(a): match the thing, never a token that mentions it).
```

The **left anchor** is what makes that claim true, and it is pinned by **P14** on the *reason*: without it the same line still blocks, but with a different message (the substitution leaves the line’s prefix in place, so the “marker” captured from : ok # EXPECTED-RED: BOB-901 is the string : okBOB-901, which then fails the id comparison). Only a reason-level assertion separates “there is no marker” from “the marker names something else”.

Caveat, stated because it is real: the match is **textual**, performed by sed over the file’s lines. A marker-shaped line inside a **heredoc** or a **multi-line string** therefore *does* count as corroboration, even though bash would never treat it as a comment. This is not independently exploitable — a reviewed table row is still required, and the row is the authority — but it means the marker half is a *corroboration* signal, not a parser-grade one.

Exit-code contract

A declaration honours exit 1 and nothing else.

Exit	Meaning	Disposition
0	The suite passed	XPASS → blocks
1	The suite's RED verdict	honoured while the declaration is valid
3	ABORT — instrument blind	blocks (not a verdict)
124	Timeout	blocks (not a verdict)
any other non-zero	crash / error	blocks (not a verdict)

A suite that aborts is reporting that *its instrument was blind*, and a blind instrument's silence is never evidence (§11.4.201(6)). Sibling suites reserve exit 3 for exactly that — see `tests/pre_build/test_bob205_danger_roots_scope.sh`. Swallowing an abort as “the expected red” would convert a blind instrument into a green one: the precise bluff this mechanism exists to prevent. **P8** is the scenario that guards this clause; before round 2 the clause was correct but unexercised, and a reviewer mutation replacing its test with `if false` survived the suite.

Boundary cases — every way a declaration can fail to be judged

This list lives here, in the repository, rather than in a session report, because a document that **binds** behaviour must be tracked where the work happens (§11.4.215).

#	Situation	Disposition
1	No table row names the suite	UNDECLARED — the gate's pre-existing rule applies untouched
2	Two or more rows name the same path	BLOCK — ambiguous
3	Row key is not a repo-relative tests/... path	BLOCK — via the sweep; it can never match a file

#	Situation	Disposition
	(bare basename, . . . , absolute)	
4	Row carries more than two fields	BLOCK — by the verdict if the path matched, by the sweep otherwise; never both (the sweep skips already-judged keys, so it is counted once)
5	Row's item id is not <PREFIX>-<N>	BLOCK
6	The suite file cannot be read	BLOCK
7	Marker absent / duplicated / naming a different id	BLOCK
8	Item id absent from docs/ workable_items.db	BLOCK
9	Store unreadable, query tool absent, or query tool <i>fails</i>	BLOCK — the failing-tool case is distinguished from “id not found”
10	Item's stored status is empty	BLOCK , without asserting a class that was never derived
11	Item's status is outside §11.4.33's closed set	BLOCK — UNKNOWN is a refusal, not a default
12	Item is CLOSED (any of the four terminal statuses)	BLOCK — stale
13	The declared suite passes	BLOCK — XPASS
14	The declared suite exits anything other than 0 or 1	BLOCK — not a verdict
15		

#	Situation	Disposition
	The declared suite was deleted or renamed	BLOCK — via the sweep; it is never executed, so it can never self-clear
16	The declared suite is quarantined or self-recursive	BLOCK — via the sweep; the loop continues before the verdict call, so its declaration would never self-clear either
17	scripts/lib/expected_red.sh is missing	The gate reports the absence loudly and installs a stub that declares nothing — every failure blocks exactly as before BOB-221
18	Nested gate invocation (BOBA_PREBUILD_NESTED)	Loop and sweep are both skipped — an honest SKIP, never a pass
19	Row is whitespace-only	IGNORED as inert — the one case that is not a refusal. It names nothing, matches nothing, and cannot silence anything. Stated explicitly rather than left as an unexplained exception to “every unresolvable input refuses” (§11.4.6)

Cases 15 and 16 were the two the round-1 review found beyond the reported set; both are closed by the same sweep, and case 16 is verified separately (a quarantined declared suite yields RAN=0 FAILED=1 HONoured=0). Case 19 was added in round 3. The sweep-side dispositions of cases 3, 4 and 5 are pinned by **P15–P17** — round 3 found them

documented but untested, and a silent continue in the sweep's shape branch left a legacy bare-basename row inert (FAILED=0, no message) with the suite still green.

Fail-closed branches (§11.4.252)

Every unresolvable input **refuses** rather than defaulting to “declared” — the full enumeration is the boundary table above. The gate additionally fail-closes on the **library itself being missing** (case 17).

Item status is read from the SSoT, never a parallel list

Open/closed is resolved from `${PROJECT_ROOT}/docs/workable_items.db` — the §11.4.93/§11.4.95 single source of truth — never from a hand-maintained list of open tickets that would rot independently.

- **CLOSED** — any status ending `Fixed.md`). §11.4.33's terminal vocabulary has **four** members — `Fixed`, `Implemented`, `Completed`, `Obsolete` — all sharing that suffix. Matching the suffix covers all four at once *and* keeps this free of any assumption about the UTF-8 arrow's encoding. Recognising only one of the four would let a declaration rot into cover behind the other three; **P9** is the scenario that guards it.
- **OPEN** — `Queued`, `In progress`, `Ready for testing`, `In testing`, `Reopened`, `Operator-blocked`.
- **anything else** — `UNKNOWN`, which is a **refusal**, not a default.
- **an empty status** — refused explicitly. The aggregate *seeds* at `CLOSED` and empty lines are skipped, so reporting that seed would state a cause that was never derived (§11.4.6). **P11** guards it.

Why the store is opened `mode=ro` — measured, not inferred

The read-only URI is the protection because **it cannot write by construction**. Measured 2026-08-27 on a scratch copy of the real WAL-mode store: an `UPDATE` through such a connection raises `OperationalError: attempt to write a readonly database`. That matters because `docs/workable_items.db` **is tracked** (`git ls-files --`

error-unmatch succeeds), so it **is** inside invariant 30’s own NO-TRACE corpus, and a real write would be caught there as a §11.4.84 quiescence violation.

An earlier revision of this document gave a **sidecar-based** rationale (“a read-write open would create `-wal/-shm` sidecars and move the tracked `.db` file’s mtime, and the NO-TRACE scan would then report the library’s read”). That causal story is **refuted in both directions**. Measured on pristine scratch copies:

open mode	sidecars while open	sidecars after clean close	.db mtime
read-write	created	removed	unchanged
mode=ro	created	left behind	unchanged

So sidecars are created in *both* modes, and it is the **read-only** open that leaves them — the reverse of the old claim. Either way they are invisible to the NO-TRACE scan, which enumerates **tracked** paths only: the sidecars are untracked and gitignored (`.gitignore docs/*.db-wal, docs/*.db-shm`). Neither open mode moved the `.db` file’s own mtime.

The original *conclusion* stands and is separately measured: **12 uncached mode=ro reads through this library left the real store’s mtime ns-identical** (2026-08-26 23:11:35.767155574 +0200 before and after).

One further caveat, for precision rather than for this library (§11.4.6): the claim “a real write would be caught by the NO-TRACE scan” holds for the **clean-close** path, where the checkpoint moves the `.db` file’s own mtime. A write that ends in an **unclean close** lands only in the untracked `-wal` sidecar, which a tracked-paths-only scan cannot see. That is moot here — `mode=ro` cannot write at all — but the scan is not a universal write-detector, and saying so keeps the guarantee honest.

Results are cached per item id — the store path derives from `PROJECT_ROOT`, which is fixed for a gate run, so `(store, id)` collapses to `id` — and the store is not touched at all when the declaration table is empty.

Public interface

Function	Contract
<code>expected_red_verdict <suite-path> <rc></code>	The single per-suite entry point the gate calls. Always exits 0; prints exactly one of UNDECLARED, HONOURED, BLOCK:<reason>. The basename is not a parameter — rows key on the repo-relative path.
<code>expected_red_unmatched_rows</code>	The post-loop table sweep. Prints one line per row this run could not evaluate; prints nothing when every row was judged, and nothing at all for an empty table.
<code>expected_red_row_shape_problem <row></code>	Shared row-shape validator. Echoes the problem and returns 0 when the row is malformed; returns 1 silently when its shape is fine.
<code>expected_red_item_status <ITEM-ID></code>	Statuses on stdout. rc 0 found, rc 1 unknown id, rc 2 store/tool unusable.
<code>expected_red_status_class <status></code>	OPEN CLOSED UNKNOWN.
<code>expected_red_note_append <label> <reason></code>	Appends a block reason to BASH_TEST_STALE_NOTE (capped at 3, then (+more)).

BLOCK:<reason> text reaches the operator verbatim in the gate's blocking line, so it names the remedy, not just the symptom.

`_EXPECTED_RED_SEEN` is the associative array of repo-relative paths the gate actually executed. **The gate loop writes it, not this library** — `expected_red_verdict` is called inside a command substitution, so anything it assigned would die with that subshell.

Adding a declaration

1. Confirm the suite genuinely exits **1** (a RED verdict, not an abort).
2. Confirm the tracked item exists and is **open**.

3. Add the in-source marker to the suite: `# EXPECTED-RED: <ITEM-ID>`.
4. Add the row to `BASH_TEST_EXPECTED_RED` in `scripts/lib/expected_red.sh`, keyed on the suite's **repo-relative path**.

Removing one

The mechanism forces this — you do not have to remember:

- the item closes → **P3** blocks, naming the stale declaration;
- the suite starts passing → **P4** blocks as an XPASS;
- the suite stops being executed at all → **P7** blocks, naming the dead row.

Remove **both** halves (the row *and* the marker) in the same commit.

The table is a ratchet

`BASH_TEST_EXPECTED_RED` must only **shrink** (§11.4.135 / §11.4.248). It ships **empty**, deliberately: an empty table means invariant 30 blocks on exactly the same set of failures it blocked on before this library landed, so landing the mechanism silences nothing by itself. Every silence is opt-in, two-place, and self-expiring.

Related

- `scripts/pre_build_verification.sh` — invariant 30, the only consumer.
- `tests/pre_build/test_bob221_expected_red_declaration.sh` — the RED that specified this mechanism; it extracts invariant 30's real loop, the real post-loop sweep, and the real blocking predicate by content marker and runs them, so it cannot drift into asserting on a private copy. It carries **no** in-source marker: retaining one while the suite is green is a latent XPASS trap and contradicts the removal rule above.
- §11.4.115(F), §11.4.120, §11.4.135, §11.4.201, §11.4.215, §11.4.224, §11.4.226, §11.4.227, §11.4.240, §11.4.248, §11.4.249, §11.4.252.

Last verified

2026-08-27 — BOB-221 suite **GREEN 19/19** (P-CONTROL + P1..P18), deterministic across 3 consecutive runs (identical verdict-line hash). **Empty-table parity** re-verified diff-identical against HEAD:scripts/pre_build_verification.sh (which contains zero expected_red references — genuinely the pre-BOB-221 rule) across three scratch scenarios.

Seventeen paired §1.1 mutations, each md5-delta-confirmed applied, each killed. Round 2 (10): basename re-key (P6), silencing the sweep (P7), honouring any non-zero exit (P8), narrowing CLOSED to the Fixed literal (P9), dropping the extra-field refusal (P10/P16), reporting the CLOSED seed (P11), honouring every undeclared suite (control needle), dropping marker/table agreement (P5), deleting the sweep from the gate, deleting the seen-set write from the loop (P1/P6). Round 3 (7): **prefix instead of exact path match (P13)**, gutting the exit-code reason text (P8), dropping the absent-marker refusal (P12), silencing the sweep's shape branch (P15/P16/P17), dropping the marker's left anchor (P14), dropping the duplicate-row check (P18), and removing the sweep's already-judged skip (caught by every counter assertion). The round-2 battery was re-run against the round-3 code: all ten still kill.

Two single-edit resistance probes on the status vocabulary confirm the property the round-1 review asked to keep: inserting a terminal status into the OPEN list is **inert** (the suffix branch is first in the case and shadows it), and altering the suffix literal degrades every terminal to **UNKNOWN**, which is a refusal and still blocks.

The carrier lesson, and why it is now structural

Three separate rounds each found a §11.4.201(7)(a) carrier, and the third one was in the *oracle*, not the product:

- round 2 — the UNKNOWN branch's message contains “closed set”, which satisfied P3/P9's case-insensitive closed alternation;
- round 3 — P8's abort alternative was satisfied by its own scenario **filename** test_zz_declared_abort.sh, so *any* declared-branch block passed P8 whatever the reason.

Renaming the file would have fixed the instance and left the class open. The suite now enforces the class mechanically, before any scenario runs: every reason alternative must be **multi-word**, and

every scenario path must be **space-free**. A filename carrier is then impossible by construction rather than by vigilance, and a future single-word alternative aborts the run with its own diagnosis.